

Pandas: Exploratory Data Analysis

PTP Program



Training Guidelines

Persyaratan dan Ekspektasi untuk Peserta Pelatihan:

1. **Prerequisite:** Peserta diharapkan sudah memiliki pengetahuan dasar tentang python dan Pandas
2. **Active Participation:** Peserta harus terlibat aktif dengan materi pelatihan, mencatat poin-poin penting dan penjelasan yang diberikan oleh instruktur.
3. **Clear Instruction:** Instruktur bertanggung jawab untuk menyampaikan materi pelatihan dengan cara yang jelas dan ringkas, memastikan semua peserta memahami topik yang dibahas.
4. **Hands-on Practice:** Sebagian besar pelatihan akan melibatkan latihan praktis yang dilakukan menggunakan **Google Colab (Jupyter Notebook)**. Anda dapat mengakses Google Colab yang tersedia di slide terakhir.

List Of Contents

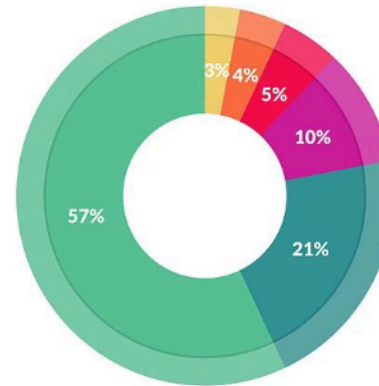
- Introduction to Data Preparation
- Data Cleaning
- Data Cleaning Practice

Data Preparation

Definisi Data Preparation

Data Preparation adalah proses dalam analisis data yang melibatkan pembersihan, transformasi, dan pengorganisasian data agar siap digunakan dalam analisis lebih lanjut.

Tujuan dari data preparation adalah untuk memastikan bahwa data berkualitas tinggi, konsisten, dan relevan untuk menjawab pertanyaan penelitian atau mendukung pengambilan keputusan.



What's the least enjoyable part of data science?

- Building training sets: 10%
- Cleaning and organizing data: 57%
- Collecting data sets: 21%
- Mining data for patterns: 3%
- Refining algorithms: 4%
- Other: 5%

Data Preparation

Manfaat Data Preparation

Berikut adalah manfaat data preparation:

1. **Efisiensi Waktu:** Dengan proses persiapan data yang efektif, data ilmuwan dapat menghabiskan lebih sedikit waktu untuk mengumpulkan, membersihkan, dan menyusun data, sehingga dapat fokus pada analisis dan penambangan data yang memberikan nilai bisnis.
2. **Pengurangan Biaya:** Persiapan data yang efisien dapat mengurangi biaya manajemen data dan analitik, karena data dikelola dan diproses dengan lebih efektif.
3. **Hasil Analisis yang Andal:** Persiapan data yang baik memastikan bahwa data yang digunakan dalam aplikasi analitik menghasilkan hasil yang dapat diandalkan dan akurat.
4. **Pengambilan Keputusan yang Lebih Baik:** Dengan data yang bersih dan terstruktur, pengambilan keputusan oleh eksekutif bisnis dan pekerja operasional menjadi lebih terinformasi dan akurat.

Data Preparation

Tahapan Data Preparation

Ada beberapa variasi dalam langkah-langkah persiapan data yang dicantumkan oleh berbagai profesional data, tetapi prosesnya biasanya melibatkan tugas-tugas berikut:

1. **Data collection:** Mengumpulkan data dari berbagai sumber, seperti basis data, file CSV, API, dll.
2. **Data Cleaning:** Mengidentifikasi, memperbaiki atau menghapus data yang kotor atau tidak akurat.
3. **Data transformation:** Mengubah data ke dalam format yang sesuai untuk analisis. Ini dapat meliputi proses normalisasi data numerik, mengonversi variabel kategori menjadi variabel numerik (encoding), dll.
4. **Data structuring:** Menyusun data dalam struktur yang logis dan mudah diakses, seperti pengelompokan atau pengurutan berdasarkan kriteria tertentu.
5. **Data Validation** adalah proses untuk memastikan bahwa data yang digunakan dalam sistem atau aplikasi memenuhi kriteria tertentu dan berkualitas tinggi.

Data Cleaning

Proses Data Cleaning

Data Cleaning adalah proses untuk mengidentifikasi, memperbaiki, atau menghapus data yang tidak akurat, tidak lengkap, tidak relevan, atau tidak konsisten dari suatu dataset.

Proses ini bertujuan untuk meningkatkan kualitas data agar lebih akurat, lengkap, dan dapat diandalkan untuk analisis, sehingga menghasilkan wawasan yang valid.

Macam-macam Proses Data Cleaning:

1. Menghapus Duplikasi Data
2. Mengidentifikasi dan Menangani Data Hilang
3. Memperbaiki Tipe Data
4. Standardizing Formats (Menyamakan Format Data)
5. Deteksi dan Menangani Outlier
6. Text Cleaning

Duplikasi Data

Menangani Duplikasi Data

Duplikasi data adalah kondisi di mana satu atau lebih entitas dalam sebuah dataset muncul lebih dari sekali. Dampak Duplikasi Data:

- **Analisis yang Tidak Akurat:** Hasil analisis bisa bias atau menyesatkan jika data yang digunakan mengandung duplikasi.
- **Peningkatan Biaya Penyimpanan:** Menyimpan data yang sama lebih dari satu kali akan menghabiskan ruang penyimpanan yang tidak perlu.
- **Kualitas Data yang Buruk:** Duplikasi dapat menyebabkan kebingungan dan mengurangi keandalan data.

Dataframe (df) yang menunjukkan data terduplikasi

id	nama	alamat	email	no_hp
1	Andi	Jl. Merdeka 1	andi@example.com	081234567890
5	Andi	Jl. Merdeka 1	andi@example.com	081234567890

```
# Cek seluruh baris yang terduplikasi
print(df.duplicated())

# Menghitung jumlah baris duplikat
df.duplicated(subset=['Email', 'No. Handphone']).sum()

# Cek duplikasi pada kolom tertentu
duplikat = df.duplicated(subset=['Email', 'No. Handphone'])

# Menghapus duplikasi
df.drop_duplicates(subset=['Email', 'No. Handphone'], inplace=True)
```


Missing Values

Definisi Missing Values

Missing values adalah nilai yang hilang atau kosong dalam sebuah dataset, yang dapat terjadi karena berbagai alasan, seperti kesalahan input, data tidak tersedia, atau kerusakan dalam proses pengumpulan data. Missing values ini biasanya ditandai dengan **NULL**, **NaN**, atau nilai kosong (empty).

Dataframe (df) yang menunjukkan data terduplikasi

id	nama	alamat	email	no_hp
1	None	Jl. Merdeka 1	andi@example.com	NaN
5	Andi	Jl. Merdeka 1	andi@example.com	081234567890

```
# Jumlah missing values per kolom
print("Jumlah missing values per kolom:")
print(df.isnull().sum())

# Persentase missing values per kolom
persentase_missing = df.isnull().sum() / len(df) * 100
print("Persentase missing values per kolom (%):")
print(persentase_missing)
```

Missing Values

Karakteristik Missing Values

Memahami karakteristik nilai yang hilang penting untuk menentukan strategi penanganan yang tepat. Untuk memahami karakteristik data yang hilang, ada tiga karakteristik nilai yang hilang yaitu MCAR, MAR, dan MNAR.

1. Missing Completely at Random (MCAR)

- MCAR mengacu pada kondisi di mana data yang hilang tidak berhubungan dengan data yang diamati maupun yang tidak diamati. Artinya, nilai yang hilang tersebar secara acak di seluruh dataset tanpa pola tertentu.

id	nama	umur	pendapatan	Kota
1	None	28	5000000	NaN
2	Andi	35	NaN	Bandung
3	Eko	30	NaN	Surabaya

Pada contoh ini, nilai yang hilang (NaN) terdapat pada kolom **Umur**, **Pendapatan**, dan **Kota**, dan tidak ada pola khusus yang mengindikasikan bahwa nilai tersebut hilang karena hubungan dengan variabel lain.

Missing Values

Karakteristik Missing Values

2. Missing at Random (MAR)

- MAR terjadi ketika kemungkinan hilangnya data terkait dengan data lain yang teramati dalam dataset, namun tidak terkait dengan data yang hilang itu sendiri. Jadi, ada hubungan antara data yang hilang dengan variabel lain yang diamati.
- Contoh:** Misalkan dalam survei kesehatan, nilai yang hilang pada kolom *Berat Badan* lebih sering terjadi pada individu yang memiliki nilai tinggi pada kolom *Usia*. Jadi, probabilitas hilangnya *Berat Badan* tergantung pada variabel *Usia* yang terlihat dalam dataset.

id	nama	umur	Berat Badan	Kota
1	None	28	70	Jakarta
2	Andi	55	NaN	Bandung
3	Eko	65	NaN	Surabaya

Pada tabel ini, nilai yang hilang (NaN) pada kolom **Berat Badan** sering muncul pada individu dengan usia lebih tinggi atau diatas 50 tahun.

Missing Values

Karakteristik Missing Values

3. Missing Not at Random (MNAR)

- MNAR terjadi ketika kemungkinan hilangnya data berkaitan langsung dengan data yang hilang itu sendiri. Ini berarti ada alasan sistematis mengapa data tersebut hilang, dan hal ini dapat menyebabkan bias pada hasil analisis.
- **Contoh:** Pada survei kesehatan, beberapa orang mungkin tidak mengisi kolom *Perokok* karena merasa sensitif terhadap informasi tersebut. Jadi, hilangnya data tersebut disebabkan oleh variabel itu sendiri yang tidak teramati.

id	nama	umur	perokok	pneumonia
1	None	28	tidak	tidak
2	Andi	55	ya	ya
3	Eko	65	NaN	ya
4	Joko	28	NaN	ya

Pada tabel ini, nilai yang hilang (NaN) pada kolom **perokok**. Asumsinya, beberapa individu mungkin enggan mengisi **perokok** karena alasan pribadi atau karena merasa tidak nyaman.

Missing Values Handling

Cara Menangani Missing Values

Pandas menyediakan beberapa metode untuk menangani nilai yang hilang dalam DataFrame. Berikut ini beberapa teknik umum:

1. Memeriksa Nilai yang Hilang:
 - Untuk mengidentifikasi nilai yang hilang dalam DataFrame, Anda dapat menggunakan metode **isnull()**, yang mengembalikan DataFrame dengan bentuk yang sama dengan True untuk nilai yang hilang dan False untuk nilai yang tidak hilang. Contoh: **df.isnull()**
2. Dropping Missing Values:
 - Anda dapat menghapus baris atau kolom yang berisi nilai yang hilang menggunakan metode **dropna()**.
 - Secara default, **dropna()** menghapus baris apa pun yang berisi setidaknya satu nilai yang hilang. Anda dapat menggunakan parameter tambahan untuk menyesuaikan perilaku, seperti menentukan **axis=1** untuk menghapus kolom, bukan baris.
3. Mengisi Nilai yang Hilang:
 - Anda dapat mengisi nilai yang hilang dengan nilai tertentu menggunakan metode **fillna()**.
 - Contoh: **df.fillna(value)**

Inconsistent Data Handling

Data Type Conversion

Sebelumnya kita telah mengetahui bahwa tipe data sangat penting untuk pemrosesan data. Untuk melakukan kalkulasi atau memanfaatkan teknik pemrosesan lanjutan, Anda harus memastikan bahwa data Anda sesuai. Tipe data Pandas berbeda dengan tipe data Python standar. Berikut ini daftar tipe data dalam pandas:

Pandas dtype	Python type	Usage
object	str or mixed	Text or mixed numeric and non-numeric values
int64	int	Integer numbers
float64	float	Floating point numbers
bool	bool	True/False values
datetime64	NA	Date and time values
timedelta[ns]	NA	Differences between two datetimes
category	NA	Finite list of text values

Mengkonversi kolom ke tipe data yang berbeda: `df['column'] = df['column'].astype(new_data_type)`

Inconsistent Data Handling

All About String

Untuk memanipulasi data string, Anda dapat melakukan beberapa hal berikut:

1. Removing Leading/Trailing Whitespaces:
 - Gunakan metode `.str.strip()` untuk menghapus spasi yang ada di awal dan akhir string.
 - Contoh: `df['column'] = df['column'].str.strip()`
2. Changing Case:
 - Ubah huruf kapital string menggunakan metode seperti `.str.lower()`, `.str.upper()`, atau `.str.title()`.
 - Contoh: `df['column'] = df['column'].str.lower()`
3. Removing or Replacing Substrings:
 - Gunakan metode `.str.replace()` untuk menghapus atau mengganti substring tertentu dalam string..
 - Contoh: `df['column'] = df['column'].str.replace('old_value', 'new_value')`
4. Memisahkan dan Mengambil Substring:
 - Pisahkan string menjadi beberapa substring menggunakan metode `.str.split()` dan ambil substring tertentu dengan metode seperti `.str.extract()` atau menggunakan ekspresi reguler.
 - Contoh: `df['new_column'] = df['column'].str.split(' ')` untuk memisahkan string berdasarkan spasi.

Inconsistent Data Handling

Datetime

Manipulasi dan pembersihan data *datetime* dalam pandas melibatkan operasi untuk memproses dan membersihkan data waktu pada kolom yang berformat tanggal. Berikut adalah beberapa tugas umum untuk manipulasi dan pembersihan data *datetime* dalam pandas:

1. Converting to Datetime:
 - gunakan `pd.to_datetime()` untuk mengonversi kolom menjadi format *datetime*.
 - Contoh: `df['column'] = pd.to_datetime(df['column'])`
2. Extracting Date Components:
 - Gunakan accessor `.dt` untuk mengekstrak komponen spesifik dari kolom *datetime*, seperti tahun, bulan, hari, jam, menit, detik, dan lain-lain.
 - Contoh: `df['year'] = df['datetime_column'].dt.year`
3. Formatting Datetime:
 - Gunakan metode `.dt.strftime()` untuk memformat nilai *datetime* ke dalam format string tertentu.
 - Contoh: `df['formatted_date'] = df['datetime_column'].dt.strftime('%Y-%m-%d')`

Inconsistent Data Handling

Datetime

4. Calculating Time Differences:

- Lakukan perhitungan pada kolom *datetime* untuk menghitung perbedaan waktu menggunakan fungsi `diff()`.
- Contoh: `df['time_diff'] = df['datetime_column'].diff()`

5. Extracting Time Intervals:

- Gunakan metode `.dt.total_seconds()` untuk mengekstrak interval waktu dalam satuan detik atau satuan lainnya dari kolom *datetime*.
- Contoh: `df['time_interval'] = df['datetime_column'].diff().dt.total_seconds()`

6. Handling Timezones:

- Gunakan metode `tz_localize()` dan `tz_convert()` untuk mengatur zona waktu pada kolom *datetime*.
- Contoh: `df['datetime_column'] = df['datetime_column'].dt.tz_localize('UTC').dt.tz_convert('America/New_York')`



External References (Practice)

Data Cleaning
practice

Visit Here